

스마트 음식 영양 분석 & 식단 추천 프로젝트

CNN 모델을 통한 사진 인식과
LLM 모델을 통한 텍스트 생성

MealStaff (2조)

박정환, 김도윤, 강성대, 이동훈, 김유진

2025-09-15

목차

- | | |
|-----------------|----------------|
| 1 개요 및 현황 | 7 업무 흐름도 |
| 2 팀원별 역할 분담 | 8 ERD |
| 3 개발 환경 | 9 인공지능 학습 (주요) |
| 4 프로젝트 일정 계획 | 10 주요 서비스 화면 |
| 5 프로젝트 구조도 | 11 시연 영상 |
| 6 메뉴 구조도 (사이트맵) | 12 소감 |

부록

- 1 기능 정의서
- 2 정책 정의서
- 3 프로젝트 구조도 (상세)
- 4 업무 흐름도 (세부)
- 5 인공지능 학습 (상세)
- 6 서비스 화면
- 7 단위 테스트
- 8 통합 테스트

1. 개요 및 현황

서비스 과제	AI 기반 음식 영양 분석 & 식단 추천 서비스
추진배경 및 목적	■ 개인별 섭취 영양 분석: 사용자의 실제 영양 섭취 상황을 분석하고 적절한 섭취량 제안 ■ 식단 추천: 사용자에게 적절한 섭취량을 바탕으로 식단 제안 ■ 사용자의 성향/기분에 따른 다양한 식단 제안: 같은 적정 섭취량이라도 사용자의 성향이나 기분에 맞춘 식단 제안으로 사용자의 만족감 제고 ■ 추천 신뢰성 강화: LLM과 머신 러닝을 사용하여 사용자의 상태에 맞는 영양소 섭취와 식단 추천 제공
주요 기능	■ 맞춤형 섭취량 제안: 사용자의 데이터에 기반한 최적의 영양 섭취량 추천 ■ 식단 제안: 사용자의 현재 영양 상태와 취향, 기분에 따른 다양한 식단 제안 ■ 개인별 식단 관리: 사용자별 영양 상태 정보와 추천받은 식단 정보 저장, 축적. 다음 식단 추천 시 더 정확한 추천 서비스 제공. ■ 개인 정보 보안: 사용자 정보에 대한 보안 강화, 개인 정보 보호.

2. 팀원별 역할 분담

팀원	역할
박정환(팀장)	프로젝트 총괄(PM), 진행, 테스터, DB, 웹페이지 구성, 회원 관리, 식단 기록
김도윤(부팀장)	주제 선정, 프로젝트 기획, 식단 추천(LLM; Ollama)
강성대	기술 총괄, 사진 분석(CNN), AI 학습/예측, 파일(복수) 업로드
이동훈	감정 기반 식단 추천(LLM; KoBERT)
김유진	사진 분석(CNN), 영양소 데이터 연동, 파일 업로드

3. 개발 환경 (1/2)

구분	항목	설명	버전
프론트 엔드	HTML/CSS	웹 페이지 구조 및 스타일링	
	JavaScript	동적 기능 및 클라이언트 측 로직 구현	
	Thymeleaf	html5 기반 템플릿 엔진, Spring Security, DB 연계 등을 위해 사용	3.1.3
백엔드	Spring Boot	웹 애플리케이션 프레임워크	3.5.4
	Java	비즈니스 로직 구현	JDK17
	Gradle	Groovy 기반 빌드 도구	8.11.1
	MariaDB	데이터 저장소 (RDBMS)	10.6
	MariaDB client	Spring Boot와 MariaDB를 연결하기 위한 도구	3.3.3
	JPA	Java에서 DB를 자동으로 매핑	3.5.2
	Spring Security	계정 비밀번호 암호화, 회원 권한 부여	6

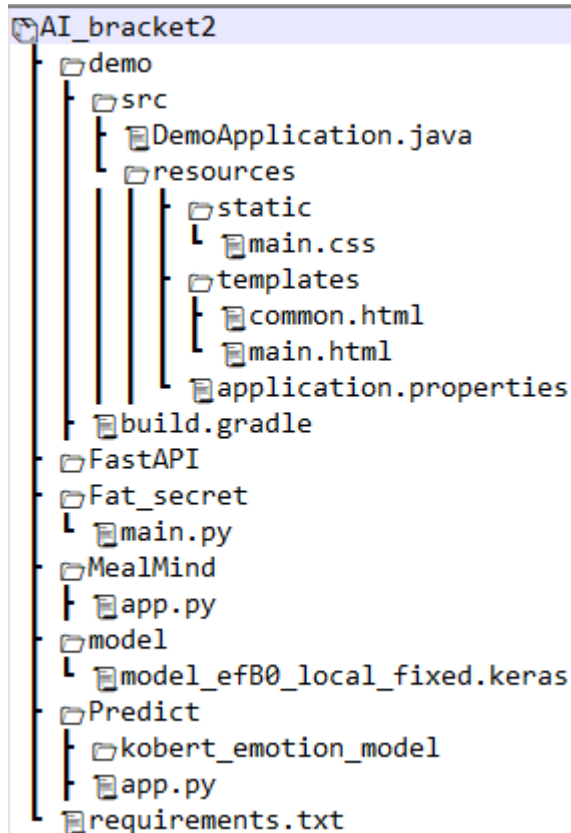
3. 개발 환경 (2/2)

구분	항목	설명	버전
백엔드	Python	머신 러닝 및 딥 러닝 예측 모델 구현	3.10.6
	FAST API	백엔드 간 데이터 통신	1.4.0
	Ollama	대규모 언어 모델을 통한 AI 서비스 (LLM)	qwen2.5:3b-instruct
	MobileNet	사진 분석 전처리 모델	V2
	EfficientNet	사진 분석을 위한 학습 모델	EfficientNetB0
	KoBERT	기분 분석을 위한 학습 모델	
	Gemini API	LLM 모델 정확도 향상을 위한 검색증강생성(RAG)용 외부 API	2.5

4. 프로젝트 일정

작업		8월														9월														
일자		18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
프로젝트 관리		월	화	수	목	금	토	일	월	화	수	목	금	토	일	월	화	수	목	금	토	일	월	화	수	목	금	토	일	월
프로젝트 착수 및 계획수립																														
개발환경 세팅																														
운영환경 세팅																														
자료 수집																														
보고																														
시스템 개발																														
분석																														
	요구사항정의																													
	개념모델구축																													
설계																														
	UI 설계																													
	시스템 설계																													
개발																														
	시스템공통																													
	웹페이지개발																													
	관리자																													
	AI 예측모델 구현																													
	단위테스트																													
	테스트실시																													
이행 및 종결	사용자테스트 및 마감																													

5. 프로젝트 구조도



📁 주요 폴더와 📄 파일

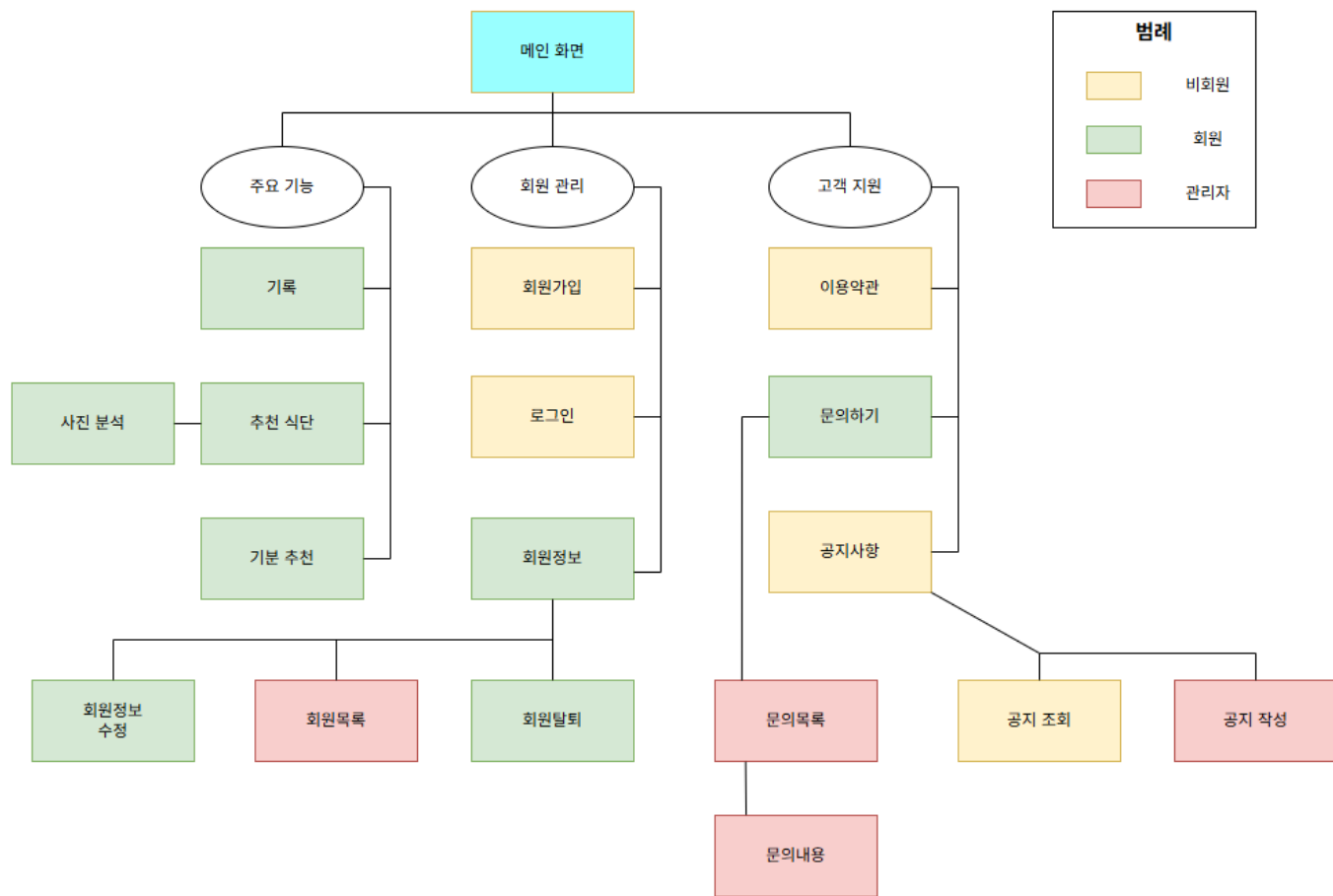
🔔 Spring Boot + Gradle

/demo/src/.../DemoApplication.java - 프로그램 실행
/demo/src/.../resources/static/main.css - 공통 스타일 (정적 문서)
/demo/src/.../resources/templates/main.html - 메인 페이지
/demo/src/.../resources/templates/common.html - 모든 문서의 header, footer에 들어가는 공통 태그
/demo/src/.../resources/application.properties - 설정 속성 정의
/demo/build.gradle - Spring Boot 프로젝트 구성, 플러그인 적용 등을 정의

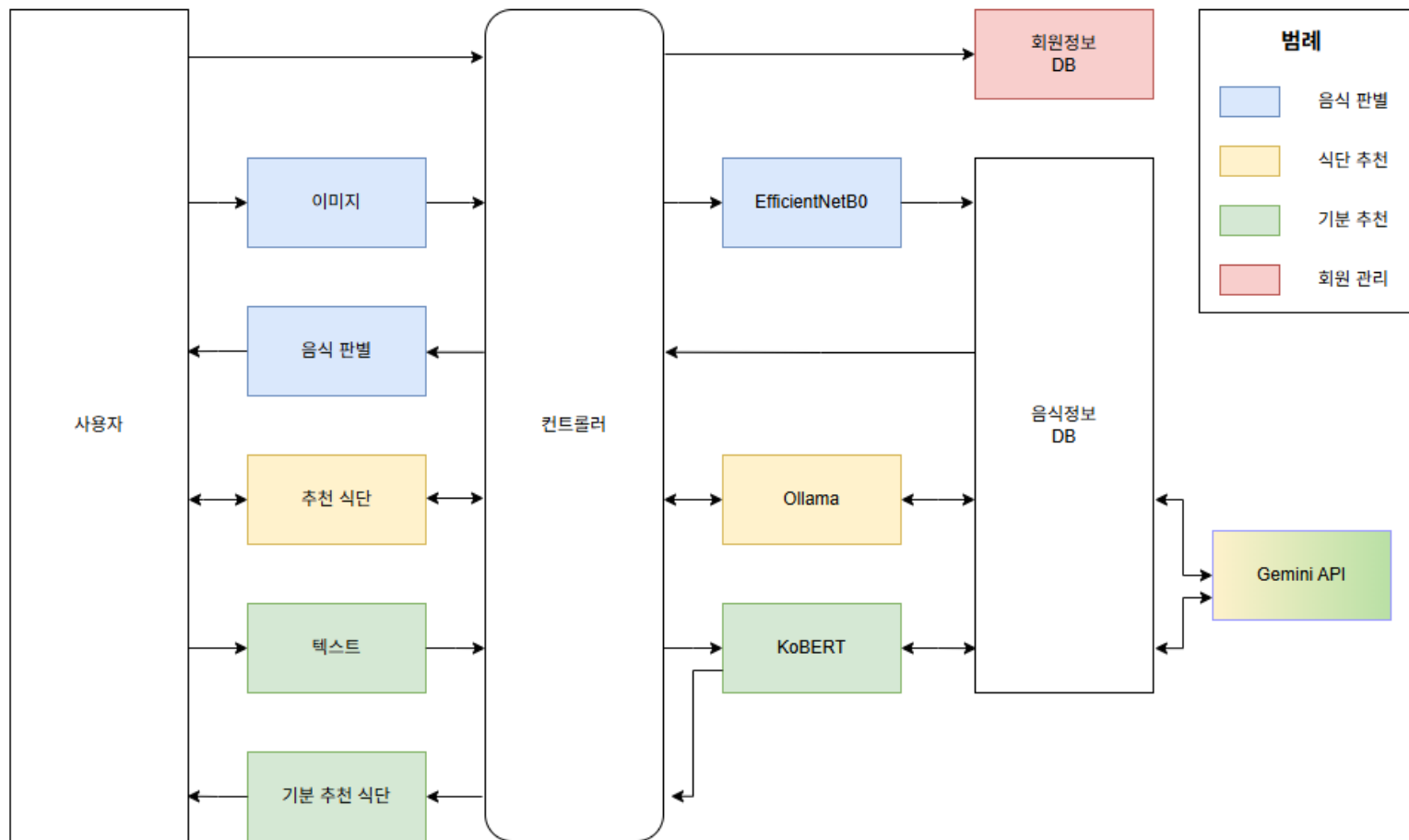
🔔 Python

/requirements.txt - pip 설치, 서버 기동 실행법을 기록한 텍스트 문서
/Fat_secret - 사진 분석 모델 (CNN)
/FastAPI - FastAPI 서버 구축, Ollama 기동
/MealMind - LLM 모델 학습
/Predict - 기분 추천
/Predict/kobert_emotion_model - KoBERT 모델

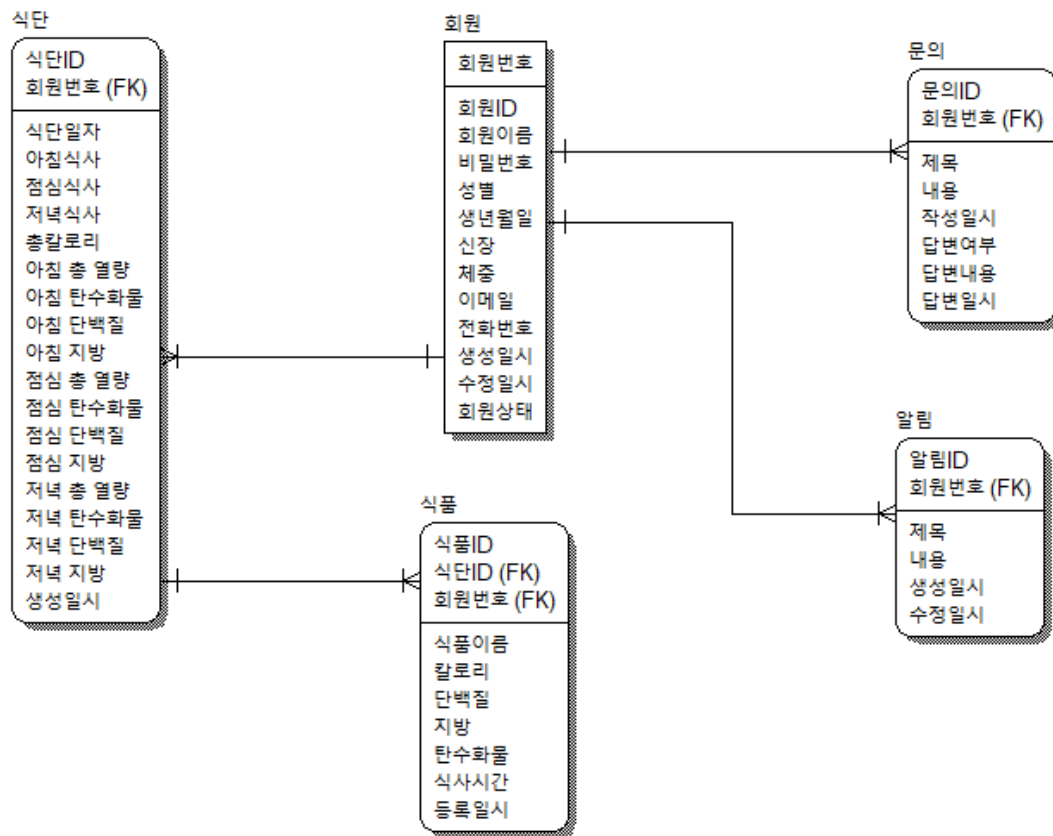
6. 메뉴 구조도 (사이트맵)



7. 업무 흐름도

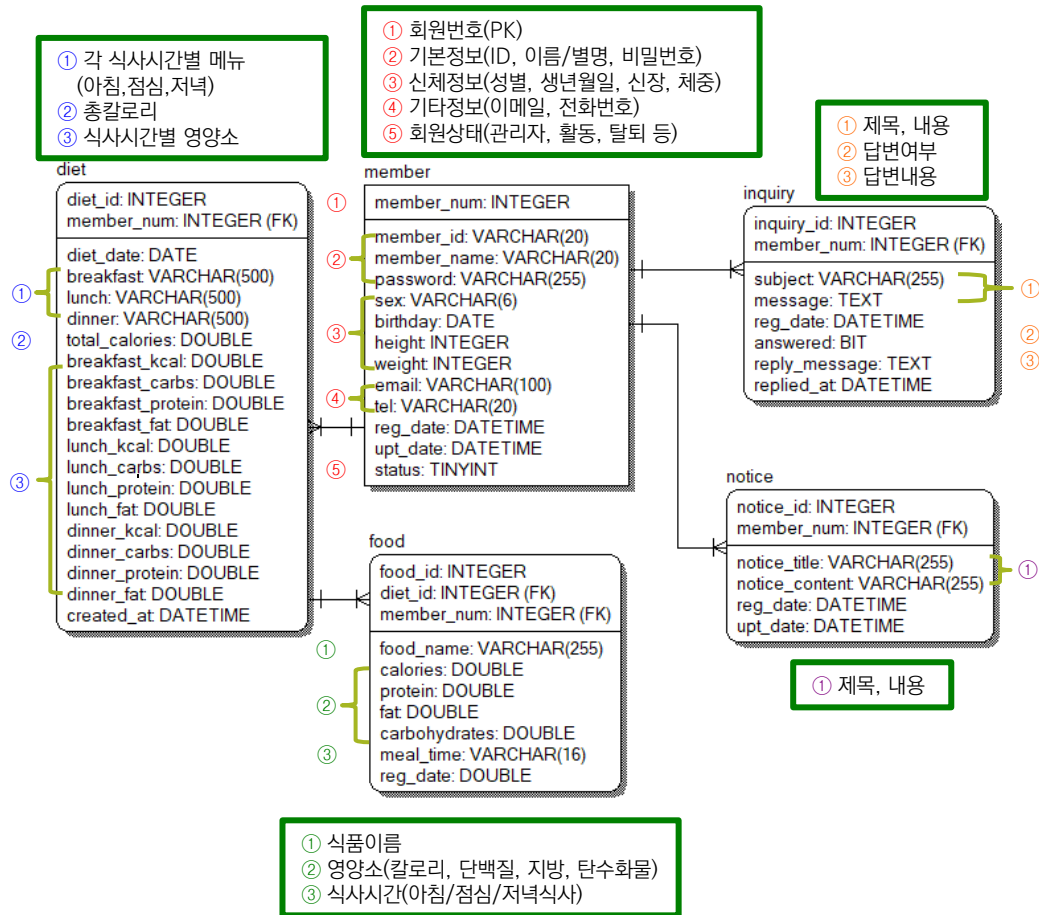


논리적 ERD



물리적 ERD

엔티티	이름	설명
member	회원	회원의 기본 정보 저장. 이메일, 전화번호를 제외한 나머지는 필수 입력.
notice	공지	사이트 공지. 관리자만 작성 가능.
inquiry	문의	회원의 문의사항을 관리자가 답변
diet	식단	하루의 아침, 점심, 저녁식사 식단 추천 정보
food	식품	식품의 각 영양소에 대한 정보



9. 인공지능 학습

EfficientNetB0 모델

■ EfficientNetB0: 경량 컨볼루션 신경망(CNN) 모델로 B0~B7중 가장 작고 기본적인 모델

- Dataset: Food Images (Food-101)
 - MobileNetV2와 동일한 데이터셋으로 진행

```
base_model = EfficientNetB0(  
    input_shape=IMG_SIZE + (3,),  
    include_top=False,  
    weights='imagenet'  
)
```

```
x = base_model.output  
x = GlobalAveragePooling2D()(x)  
x = Dropout(0.4)(x)  
predictions = Dense(num_classes, activation='softmax')(x)
```

- 모델 구성: 전이 학습 적용+드롭아웃(Dropout) 레이어 추가
- 전과 같이 전이 학습을 적용해서 학습 시간 단축 및 효율성 증가
 - 과적합 완화를 위해서 드롭아웃 비율을 0.4로 변경

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])  
EPOCHS = 20
```

```
callbacks = [  
    tf.keras.callbacks.EarlyStopping(  
        monitor='val_loss',  
        patience=5,  
        restore_best_weights=True  
    )  
]
```

학습 환경 설정:

- 옵티마이저: Adam 옵티마이저 사용
- 학습률: 기본 학습률(0.001)
- 에포크 20회로 설정
- EarlyStopping을 적용하여 과적합 방지

• 학습 진행 및 결과

```
Epoch 1/20  
2525/2525 ----- 255s 90ms/step - accuracy: 0.4422 - loss: 2.4137 - val_accuracy: 0.6487 - val_loss: 1.3763  
Epoch 2/20  
2525/2525 ----- 218s 80ms/step - accuracy: 0.6220 - loss: 1.4527 - val_accuracy: 0.6694 - val_loss: 1.2831  
Epoch 3/20  
2525/2525 ----- 179s 71ms/step - accuracy: 0.6424 - loss: 1.3622 - val_accuracy: 0.6750 - val_loss: 1.2489  
Epoch 4/20  
2525/2525 ----- 195s 77ms/step - accuracy: 0.6533 - loss: 1.3118 - val_accuracy: 0.6804 - val_loss: 1.2292  
Epoch 5/20  
2525/2525 ----- 181s 72ms/step - accuracy: 0.6551 - loss: 1.2934 - val_accuracy: 0.6815 - val_loss: 1.2268  
Epoch 6/20  
2525/2525 ----- 195s 77ms/step - accuracy: 0.6625 - loss: 1.2762 - val_accuracy: 0.6854 - val_loss: 1.2151  
Epoch 7/20  
2525/2525 ----- 189s 75ms/step - accuracy: 0.6646 - loss: 1.2633 - val_accuracy: 0.6885 - val_loss: 1.2065  
Epoch 8/20  
2525/2525 ----- 207s 77ms/step - accuracy: 0.6670 - loss: 1.2548 - val_accuracy: 0.6896 - val_loss: 1.2079  
Epoch 9/20  
2525/2525 ----- 190s 75ms/step - accuracy: 0.6661 - loss: 1.2525 - val_accuracy: 0.6900 - val_loss: 1.2067  
Epoch 10/20  
2525/2525 ----- 206s 77ms/step - accuracy: 0.6671 - loss: 1.2451 - val_accuracy: 0.6898 - val_loss: 1.2065  
Epoch 11/20  
2525/2525 ----- 192s 76ms/step - accuracy: 0.6661 - loss: 1.2532 - val_accuracy: 0.6914 - val_loss: 1.2043  
Epoch 12/20  
2525/2525 ----- 196s 78ms/step - accuracy: 0.6655 - loss: 1.2473 - val_accuracy: 0.6883 - val_loss: 1.2103  
Epoch 13/20  
2525/2525 -----  
Epoch 15/20  
2525/2525 ----- 197s 75ms/step - accuracy: 0.6681 - loss: 1.2352 - val_accuracy: 0.6929 - val_loss: 1.2089  
Epoch 16/20  
2525/2525 ----- 208s 78ms/step - accuracy: 0.6661 - loss: 1.2438 - val_accuracy: 0.6879 - val_loss: 1.2133
```

테스트 데이터셋의 손실(Loss): 1.2043
테스트 데이터셋의 정확도(Accuracy): 0.6914

- 11번째 에포크 이후부터 과적합이 시작됨에 따라 Early Stopping으로 학습이 중단됨
- 검증 정확도는 0.6914

9. 인공지능 학습

EfficientNetB0 모델

- 미세 조정

```
callbacks2 = [  
    EarlyStopping(  
        monitor='val_loss',  
        patience=5,  
        restore_best_weights=True  
    ),  
    ReduceLROnPlateau(  
        monitor='val_loss',  
        factor=0.2,  
        patience=3,  
        min_lr=0.000001  
    )  
]
```

```
base_model.trainable = True  
  
EPOCHS2=50  
  
fine_tune_at = -20  
for layer in base_model.layers[:fine_tune_at]:  
    layer.trainable = False  
  
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

- 모델의 마지막 20개 레이어만 가중치 동결 해제(나머지 레이어는 반복문으로 가중치 동결 및 학습 제외)
 - 초기 레이어는 고정함으로서 모델의 안정성 유지
 - 마지막 레이어는 새로운 데이터에 맞춰서 미세하게 조정
- 콜백 함수에 EarlyStopping, ReduceLROnPlateau를 추가
 - 검증 손실이 특정 기간(patience=3) 동안 개선되지 않으면 학습률을 동적으로 감소

- 학습 진행 및 결과

```
Epoch 1/50  
252s/252s ----- 266s 95ms/step - accuracy: 0.4716 - loss: 2.1806 - val_accuracy: 0.6564 - val_loss: 1.3419 - learning_rate: 1.0000e-05  
Epoch 2/50  
252s/252s ----- 214s 82ms/step - accuracy: 0.6070 - loss: 1.5071 - val_accuracy: 0.6819 - val_loss: 1.2312 - learning_rate: 1.0000e-05  
Epoch 3/50  
252s/252s ----- 187s 74ms/step - accuracy: 0.6418 - loss: 1.3381 - val_accuracy: 0.6935 - val_loss: 1.1817 - learning_rate: 1.0000e-05  
Epoch 4/50  
252s/252s ----- 205s 81ms/step - accuracy: 0.6592 - loss: 1.2763 - val_accuracy: 0.7025 - val_loss: 1.1514 - learning_rate: 1.0000e-05  
Epoch 5/50  
252s/252s ----- 197s 78ms/step - accuracy: 0.6733 - loss: 1.2064 - val_accuracy: 0.7090 - val_loss: 1.1278 - learning_rate: 1.0000e-05  
Epoch 6/50  
252s/252s ----- 202s 80ms/step - accuracy: 0.6807 - loss: 1.1742 - val_accuracy: 0.7139 - val_loss: 1.1075 - learning_rate: 1.0000e-05  
Epoch 7/50  
252s/252s ----- 196s 77ms/step - accuracy: 0.6918 - loss: 1.1378 - val_accuracy: 0.7183 - val_loss: 1.0921 - learning_rate: 1.0000e-05  
Epoch 8/50  
252s/252s ----- 202s 80ms/step - accuracy: 0.6980 - loss: 1.1104 - val_accuracy: 0.7205 - val_loss: 1.0785 - learning_rate: 1.0000e-05  
Epoch 9/50  
252s/252s ----- 240s 75ms/step - accuracy: 0.7044 - loss: 1.0877 - val_accuracy: 0.7235 - val_loss: 1.0664 - learning_rate: 1.0000e-05  
Epoch 10/50  
252s/252s ----- 216s 80ms/step - accuracy: 0.7147 - loss: 1.0509 - val_accuracy: 0.7266 - val_loss: 1.0553 - learning_rate: 1.0000e-05  
Epoch 11/50  
252s/252s ----- 246s 74ms/step - accuracy: 0.7173 - loss: 1.0406 - val_accuracy: 0.7283 - val_loss: 1.0460 - learning_rate: 1.0000e-05  
Epoch 12/50  
252s/252s ----- 203s 80ms/step - accuracy: 0.7192 - loss: 1.0174 - val_accuracy: 0.7307 - val_loss: 1.0378 - learning_rate: 1.0000e-05  
Epoch 13/50  
...  
Epoch 49/50  
252s/252s ----- 191s 75ms/step - accuracy: 0.8152 - loss: 0.6556 - val_accuracy: 0.7648 - val_loss: 0.9254 - learning_rate: 1.0000e-05  
Epoch 50/50  
252s/252s ----- 212s 79ms/step - accuracy: 0.8154 - loss: 0.6430 - val_accuracy: 0.7653 - val_loss: 0.9239 - learning_rate: 1.0000e-05
```

```
632/632 ----- 34s 54ms/step - accuracy: 0.7670 - loss: 0.9232  
테스트 데이터셋의 손실(Loss): 0.9239  
테스트 데이터셋의 정확도(Accuracy): 0.7653
```

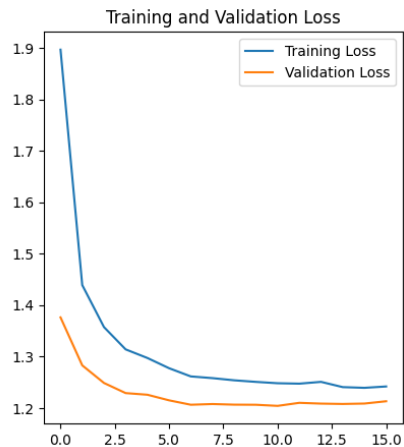
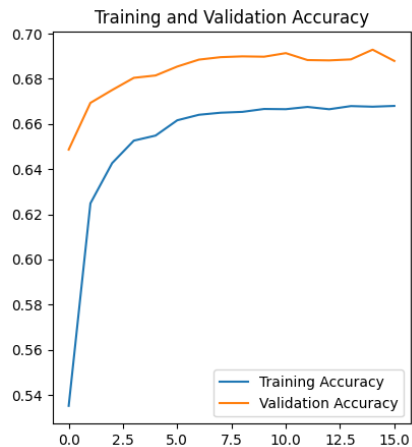
- 학습/검증 데이터에 대한 정확도가 꾸준히 증가함(손실값 또한 꾸준히 감소)
- 검증 데이터에 대한 정확도: 0.7653
- 검증 데이터에 대한 손실: 0.9239
 - 초기 학습 결과보다 향상된 결과를 얻어냄

9. 인공지능 학습

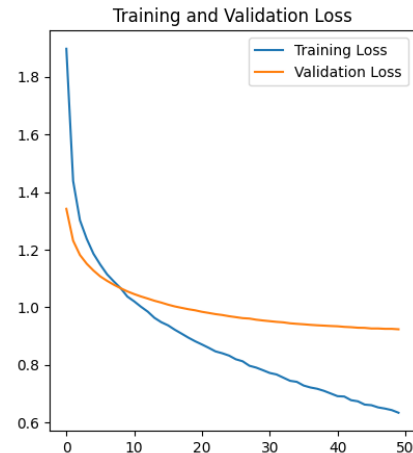
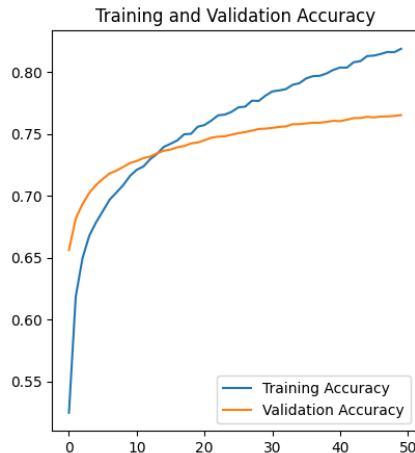
EfficientNetB0 모델

- 학습 결과에 대한 그래프

▶ 1차 학습 결과



▶ 미세 조정 결과



Ollama (LLM)

■ 사용 모델 & API

- LLM 엔진: Ollama — qwen2.5:3b-instruct
- 호출 조건: POST /recommend?live=true|false (기본 false)
- live=true → LLM 호출 → 응답 JSON 파싱
- live=false 또는 실패/포맷 불일치 → Fallback JSON(동일 스키마)
- LLM API(내부): POST {OLLAMA_HOST}/api/generate
- Body: { model: OLLAMA_MODEL, prompt, stream:false, options:{ temperature:0.8, repeat_penalty:1.1, seed } }
- Timeout: HTTP_TIMEOUT(기본 30s)
- 환경변수: OLLAMA_HOST= http://127.0.0.1:11434, OLLAMA_MODEL= qwen2.5:3b-instruct

```
import os
import time
import httpx
from typing import Optional

OLLAMA_HOST = os.getenv("OLLAMA_HOST", "http://127.0.0.1:11434")

async def is_alive() -> bool:
    url = f"{OLLAMA_HOST}/api/tags"
    try:
        async with httpx.AsyncClient(timeout=2.0) as client:
            resp = await client.get(url)
            return resp.status_code == 200
    except Exception:
        return False

async def generate_with_ollama(prompt: str, system: Optional[str], model: str, stream: bool) -> str:
    """
    Ollama /api/generate 호출 (stream=False 기본).
    """
    url = f"{OLLAMA_HOST}/api/generate"
    payload = {
        "model": model,
        "prompt": prompt if system is None else f"System: {system}\n\nUser: {prompt}",
        "stream": stream,
    }

    t0 = time.perf_counter()
    async with httpx.AsyncClient(timeout=60.0) as client:
        r = await client.post(url, json=payload)
        r.raise_for_status()
        data = r.json()

    # /api/generate 는 stream=false일 때 전체 결과가 "response" 필드로 옴
    content = data.get("response", "")
    latency_ms = int((time.perf_counter() - t0) * 1000)
    return content, latency_ms
```


Ollama (LLM)

■ 모델 개요

- DB 프로필(gender, height_cm, weight_kg)로 하루 식단 자동 추천
- Mifflin BMR $\rightarrow$ TDEE $\times 1.375 \rightarrow$ 목표 kcal 산출
- 30/40/30 끼니 배분, 영양(단/탄/지/섬유) 계산
- live=true면 LLM, 아니면 Fallback 즉시 생성

■ 모델 흐름(엔진)

- 입력 정규화 $\rightarrow$ BMR/TDEE 계산 $\rightarrow$ 끼니 목표 kcal 분배
- LLM 호출(옵션) $\rightarrow$ 식단 JSON 수신
- 결과 정규화(단위 제거·리스트화) $\rightarrow$ 영양 합산
- 목표 대비 스케일링(± 200 kcal), reason 생성

```
def _normalize_gender(g: str) -> str:
    if not g:
        return "female"
    s = g.strip().lower()
    if s in ("남", "m", "male", "man", "boy"):
        return "male"
    if s in ("여", "f", "female", "woman", "girl"):
        return "female"
    return "female"

def _bmr_mifflin(gender: str, height_cm: float, weight_kg: float, age: int = 21) -> float:
    g = _normalize_gender(gender)
    if g == "male":
        return 10 * weight_kg + 6.25 * height_cm - 5 * age + 5
    return 10 * weight_kg + 6.25 * height_cm - 5 * age - 161

def _activity_factor() -> float:
    return 1.375 # light

def calc_goal_calories(req: RecommendRequest) -> int:
    bmr = _bmr_mifflin(req.gender, req.height_cm, req.weight_kg, age=21)
    tdee = bmr * _activity_factor()
    return int(round(tdee / 10.0) * 10)

def _split_calories(total: int) -> Dict[str, int]:
    b = int(round(total * 0.30))
    l = int(round(total * 0.40))
    d = total - b - l
    return {"breakfast": b, "lunch": l, "dinner": d}

def _compute_fiber_g(kcal: int) -> int:
    return max(2, int(round(kcal / 1000.0 * 14)))

def _compute_nutrients(kcal: int, protein_ratio=0.25, carbs_ratio=0.50, fat_ratio=0.25) -> Dict[str, int]:
    protein_g = int(round(kcal * protein_ratio / 4.0))
    carbs_g = int(round(kcal * carbs_ratio / 4.0))
    fat_g = int(round(kcal * fat_ratio / 9.0))
    fiber_g = _compute_fiber_g(kcal)
    return {"protein_g": protein_g, "carbs_g": carbs_g, "fat_g": fat_g, "fiber_g": fiber_g}

def _sum_nutrients(a: Dict[str, int], b: Dict[str, int]) -> Dict[str, int]:
    return {
        "protein_g": a.get("protein_g", 0) + b.get("protein_g", 0),
        "carbs_g": a.get("carbs_g", 0) + b.get("carbs_g", 0),
        "fat_g": a.get("fat_g", 0) + b.get("fat_g", 0),
        "fiber_g": a.get("fiber_g", 0) + b.get("fiber_g", 0),
    }
```

9. 인공지능 학습

Ollama (LLM)

■ 예외 처리 & 품질

- LLM 실패/포맷 불일치/타임아웃 → Fallback JSON(동일 스키마) 반환
- 표시 전 스키마 검증 & 숫자 파싱
- 지표: 목표 kcal 오차($|\Delta| \leq 200$), Fallback 비율, 응답시간(LLM on/off), 스타일 일관성

```
def _cohere_or_replace(meal_key: str, menu: str, rnd: random.Random) -> str:
    items = [t.strip() for t in _SPLIT_PATTERN.split(menu) if t.strip()]
    score = _coherence_score(items)
    if score >= 0.67:
        return ", ".join(items[:4]) # 적절
    # 지배 스타일 선택
    tags = [t for t in (_style_of_item(x) for x in items) if t != "unknown"]
    dominant = None
    if tags:
        from collections import Counter
        dominant = Counter(tags).most_common(1)[0][0]
    # fallback 후보 중 같은 스타일 우선 선택
    candidates = FALLBACK_BANK[meal_key][:]
    rnd.shuffle(candidates)
    def menu_style(m: str) -> str:
        its = [t.strip() for t in _SPLIT_PATTERN.split(m) if t.strip()]
        tags = [t for t in (_style_of_item(x) for x in its) if t != "unknown"]
        if not tags: return "unknown"
        from collections import Counter
        return Counter(tags).most_common(1)[0][0]
    if dominant:
        for c in candidates:
            if menu_style(c) == dominant:
                return c
    # 못 찾으면 아무거나(이미 후보는 모두 내부적으로 일관성 있음)
    return rnd.choice(candidates)
```

Ollama (LLM)

■ LLM 프롬프트 요지(규칙)

- 한 끼 한 가지 스타일(한식 밥상 / 면 / 빵·양식 / 샐러드 볼 / 일식 중 1)만 사용
- 끼니 간 메인 중복 금지, 한국 사용자 친숙한 1~4개 아이템로 menu 구성
- 목표 kcal ± 200 허용, JSON만 반환

```
Return:
You are a skilled diet planner for Korean users. Return STRICT JSON only.

Inputs:
- gender={gender}, height_cm={req.height_cm}, weight_kg={req.weight_kg}
Rules:
- Target total = {goal} kcal ( $\pm 200$  kcal OK). Split: breakfast 30%, lunch 40%, dinner 30%.
- Each meal MUST be ONE style among:
  (a) Korean rice set(밥+국+반찬), (b) Noodle set, (c) Bread/Western set,
  (d) Salad bowl, (e) Japanese set. Do NOT mix across styles within the SAME meal.
- Encourage VARIETY across meals (Korean/Western/Japanese/Salad/Pasta/덮밥 등).
- Do NOT repeat the same main dish across breakfast/lunch/dinner.
- The example below shows JSON SHAPE ONLY. DO NOT COPY ITS MENU.
- Each "menu" has 1~4 items, comma-separated. Use names familiar to Korean users.
```

9. 인공지능 학습

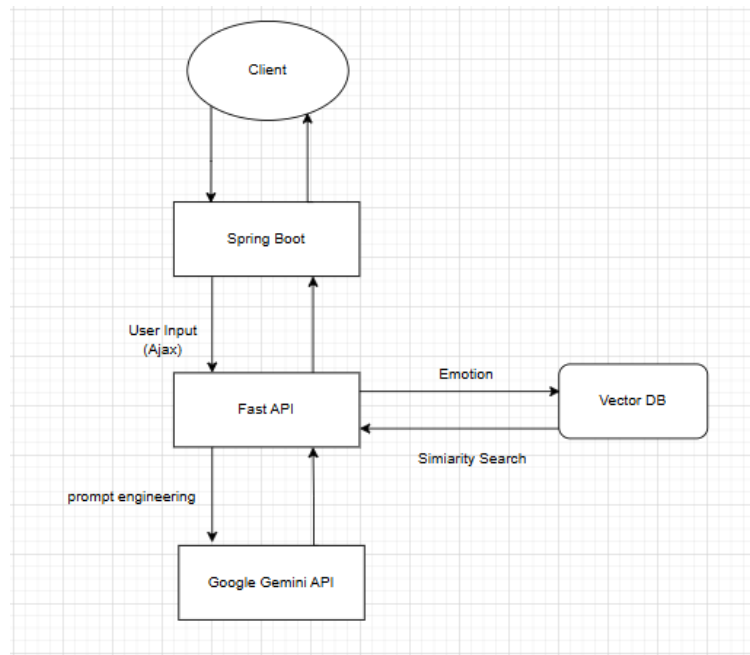
KoBERT (LLM)

■ 감정 분석 & 음식 추천 서비스 흐름

1. 텍스트 기반 감정 분석 → 감정별 음식 추천
2. 사전 학습 KoBERT 모델 활용
3. Gemini LLM API 연동

■ 모델 및 토크나이저

- 모델: KoBERT (BERTForSequenceClassification)
- 라벨: 공포, 놀람, 분노, 슬픔, 중립, 행복, 혐오 (7개)
- 토크나이저: 모델 전용 AutoTokenizer
- 사전 학습 모델 다운로드 후 추론만 진행



9. 인공지능 학습

KoBERT (LLM)

- 텍스트 감정 분석의 이해

- KoBERT 모델 활용

한국어에 최적화된 BERT 모델인 KoBERT를 사용했습니다.

이 모델은 텍스트를 분석하여 '공포, 놀람, 분노, 슬픔, 중립, 행복, 혐오' 7가지 감정 중 하나로 분류합니다.

모델은 이미 학습이 완료된 상태이므로, 별도의 학습 없이 바로 추천 서비스에 활용할 수 있습니다.

9. 인공지능 학습

KoBERT (LLM)

■ 데이터 전처리

● 모델이 이해하는 언어

토큰나이저(Tokenizer): 문장을 숫자로 변환

사용자가 입력한 문장을 모델이 이해할 수 있는 숫자(토큰) 배열로 변환합니다.

모든 문장 길이를 256자로 통일하고(padding & truncation), 모델이 읽을 수 있는 최종 데이터 형식인 **텐서(Tensor)**로 만듭니다.

```
encoding = tokenizer(  
    text,  
    truncation=True,  
    padding="max_length",  
    max_length=256,  
    return_tensors="pt"  
)
```

9. 인공지능 학습

KoBERT (LLM)

■ 정확도 향상 전략

● 예측의 신뢰도를 높이는 방법

규칙 기반 보정:

모델이 예측한 감정 결과에 키워드와 규칙을 적용하여 정확도를 높였습니다.

예를 들어, "너무 무섭지 않아"라는 문장에서 '무섭'이라는 키워드를 감지하고, 뒤에 따라오는 '않아'와 같은 부정어를 인식하여 정확한 감정(중립이나 슬픔)으로 보정합니다.

아래 코드가 감정 예측과 보정 로직을 보여줍니다.

예측 보정 로직



```
# 라벨 클래스 및 키워드
label_classes = ['공포', '놀람', '분노', '슬픔', '중립', '행복', '혐오']
emotion_keywords = {
    "분노": ["짜증", "열받", "빡치", "화나", "어이없", "화가"],
    "슬픔": ["슬퍼", "우울", "상심", "속상", "눈물"],
    "행복": ["행복", "좋아", "즐거", "기쁘", "신남"],
    "공포": ["무서", "겁나", "두려", "소름"],
    "놀람": ["놀라", "헉", "어머", "와우"],
    "혐오": ["역겹", "싫어", "구역질", "짜증"]
}
negation_words = ["안", "못", "없", "아니", "지않"]
```

```
def adjust_emotion_by_keywords(text: str, probs: torch.Tensor) -> torch.Tensor:
    text_proc = re.sub(r"\s+", "", text)
    for idx, label in enumerate(label_classes):
        for kw in emotion_keywords.get(label, []):
            if kw in text_proc:
                neg = any(text_proc.find(nw + kw) != -1 for nw in negation_words)
                if neg:
                    probs[idx] *= 0.3
                else:
                    probs[idx] += 0.3

    probs = probs / probs.sum()
    return probs
```

KoBERT (LLM)

■ 음식 추천 로직

● 감정과 음식을 연결하는 기술

벡터 데이터베이스 (Vector DB):

각 감정별로 미리 정의된 설명을 **숫자 형태(벡터)**로 저장해 둔 데이터베이스입니다.

RAG (검색 증강 생성): 사용자의 감정(예: '행복')을 분석하면, 이 감정과 가장 유사한 벡터를 데이터베이스에서 검색합니다.

검색된 감정 설명(예: "행복은 만족감, 기쁨을 느끼는 감정입니다.")을 Gemini에게 전달하여 프롬프트를 만듭니다.

아래 코드가 Gemini API를 호출하여 음식 추천을 받는 부분입니다. Gemini는 이 정보를 바탕으로 감정에 맞는 음식 3가지를 추천해 줍니다.

```
print(f"예측된 감정: {emotion}, 검색된 문서 라벨: {retrieved_emotion_label}")
print(f"검색된 문서 내용:\n{context}")

prompt = (
    f"다음은 '{retrieved_emotion_label}' 감정에 대한 배경 설명이야:\n{context}\n\n"
    f"이 감정에 맞는 음식을 세 가지 추천해줘. "
    f"각 음식은 번호를 붙이고, 음식 이름과 간단한 이유를 덧붙여줘."
)
```


10. 주요 서비스 화면

프로젝트명	스마트 음식 영양 분석 & 식단 추천 프로젝트	화면명	메인 페이지	화면 ID	DP-1010
위 치	홈	파일명	main.html	버전/날짜	2025-09-12

MealMind에 오신 것을 환영합니다.	A4	메인 메뉴	사진 분석	추천 식단	기본 추천	기록	회원정보	로그인	회원가입
D1	A1	A2	A3						



MEALMIND

AI 기반 맞춤형 음식 추천 서비스

사진으로 음식의 영양소를 분석하고, 개인 맞춤형 건강한 식단을 추천해드립니다.

- ✓ 음식 사진을 찍어 영양소를 즉시 분석
- ✓ 개인 체질에 맞는 식단 추천
- ✓ 일일 영양소 섭취량 기록 및 관리

건강한 식습관 형성을 위한 스마트한 솔루션을 경험해보세요.

Description	
1	서비스의 첫 화면
D1	로그인하면 환영 문구가 달라짐

Action		
번호	버튼명	기능
A1	회원정보	회원 정보 표시
A2	로그인	회원 로그인. 로그아웃 상태에서는 로그인 버튼이 됨.
A3	회원가입	회원 가입. 회원은 서비스를 이용 가능.
A4	메뉴 버튼	각 기능별 페이지로 이동
A5	슬라이드	여러 장의 음식 이미지를 슬라이드 형식으로 보여줌
A6	고객지원 버튼	각 운영사항에 대해 알리는 메뉴 페이지로 이동

고객지원

이용약관
문의하기
공지사항

A6

연락처

주소: 경기도 수원시 팔달구 중부대로 104
전화: 031-234-5678

10. 주요 서비스 화면

프로젝트명	스마트 음식 영양 분석 & 식단 추천 프로젝트	화면명	음식 분석 결과	화면 ID	DP-2011
위 치	홈 > 사진 분석 > 분석하기	파일명	photo/analyse.html	버전/날짜	2025-09-04

가가가님 환영합니다!

메인 메뉴 사진 분석 추천 식단 기본 추천 기록

회원정보 로그아웃 회원가입

음식 분석기

분석 결과



예상 음식: 곱창전골
신뢰도: 100.00%
칼로리: 532.65
탄수화물: 26.89
단백질: 38.35
지방: 34.73

식사시간을 선택해주세요.

다시 분석

음식 저장

localhost:8080 내용:
저장되었습니다!

A3

확인

Action		
번호	버튼명	기능
A1	다시 분석	이번 분석 결과를 저장하지 않고, 다른 사진으로 음식 분석을 한다.
A2	음식 저장	분석 결과가 나온 음식을 식단 추천을 위해 저장한다. V1에서 지정한 식사시간이 있을 경우, 그 정보도 함께 저장한다.
A3	음식 저장 확인	음식 정보가 저장되었음을 확인. 버튼을 누르면 식단 추천 화면으로 넘어간다.

Code	
C1	식사시간(아침/점심/저녁)

10. 주요 서비스 화면

프로젝트명	스마트 음식 영양 분석 & 식단 추천 프로젝트	화면명	식단추천 결과	화면 ID	DP-3020
위 치	홈 > 식단추천 > 추천결과	파일명	meal/result.html	버전/날짜	2025-09-12

가가가님 환영합니다!

메인 메뉴 사진 분석 추천 식단 기본 추천 기록

회원정보 로그아웃 회원가입

D1 **가가가님을 위한 다음 식사 (mealLabel)**

2025-09-12 금요일

D2 ☒ 균형 잡힌 영양 섭취를 위한 추천 식단입니다.

D3

D4 **일일 영양소 현황**

⚠ 주의: 일일 권장량을 초과하여 섭취하셨습니다. 다이어트를 고려해보세요.

권장 칼로리: 2171 kcal

현재까지 섭취: 1628 kcal

탄수화물: 203.5g / 271.3g

단백질: 101.8g / 135.7g

지방: 45.2g / 60.3g

D5

아침 추천 메뉴

현미밥, 달걀찜, 시금치나물, 김치

543 kcal

탄수화물 67.9g · 단백질 34.0g · 지방 15.0g

상세 영양정보:

현미밥

217.1kcal, 탄33.9g, 단6.8g, 지1.5g

달걀찜

162.8kcal, 탄6.8g, 단17.0g, 지6.0g

시금치나물

108.5kcal, 탄20.4g, 단6.8g, 지4.5g

김치

54.3kcal, 탄6.8g, 단3.4g, 지3.0g

추천 이유: 일일 권장량 2171 kcal 중 543 kcal가 남아있어 아침로 543 kcal를 권장합니다. 균형잡힌 영양소 섭취를 위해 탄수화물, 단백질, 지방을 고르게 배분했습니다.

A1

이 식단 저장

식단 화면으로 돌아가기

A2

Description	
D1	회원의 닉네임을 불러온다.
D2	하루 권장 섭취량을 넘지 않을 경우 추천 식단 알림 표시
D3	하루 권장 섭취량을 넘을 경우, 다이어트 권고 알림 표시
D4	현재 하루 권장 섭취량과, 지금까지 섭취량을 비교하여 표시
D5	추천 식단(메뉴, 영양성분, 칼로리) 표시.

Action		
번호	버튼명	기능
A1	식단 저장	추천 식단을 저장하고 기록 페이지로 이동한다.
A2	돌아가기	식단 화면으로 돌아가기

10. 주요 서비스 화면

프로젝트명	스마트 음식 영양 분석 & 식단 추천 프로젝트	화면명	식단 기록 (캘린더)	화면 ID	DP-3100
위 치	홈 > 기록(캘린더)	파일명	meal/calendar.html	버전/날짜	2025-09-12

가가가님 환영합니다!

메인 메뉴 사진 분석 추천 식단 기본 추천 기록

회원정보 로그아웃 회원가입

내 식단 기록

C1

☒ 캘린더 ☐ 날짜별

C2

2025년 09월 이동

A1

일	월	화	수	목	금	토
	1	2 2170 kcal	3	4	5 2170 kcal	6
7	8	9	10	11 880.8 kcal	12 1620.7 kcal	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

Description	
1	회원의 식단 기록을 달력 형태로 보여준다.

Action		
번호	버튼명	기능
A1	(연,월 이동)	선택한 년, 월로 이동
A2	상세 기록	기록이 있는 날짜에 식단의 총 열량이 나오는데, 이를 클릭하면 개별 날짜의 추천 식단 기록 페이지로 이동한다.

Code	
C1	캘린더/날짜별 화면 전환
C2	연도, 월 이동

고객지원
이음약관
문의하기
공지사항

연락처
주소: 경기도 수원시 팔달구 중부대로 104
전화: 031-234-5678
이메일: contact@mealstaff.com

© 2025 MealStaff. All rights reserved.

시연 영상

<https://www.youtube.com/watch?v=MTN5ArbfuFQ>

소감

이 프로젝트는 제가 지난 8개월 간 소프트웨어 개발자로서 교육을 받으며 수행했던 마지막 프로젝트입니다.

팀장을 맡다 보니 막상 AI 기술보다 다른 작업에 대부분의 시간을 보낸 것이 개인적으로는 아쉬웠습니다. 그래도 평소에 관심있던 AI 기술을 주제로 결과물을 완성한 것은 앞으로의 경력에 큰 자산이 될 것이라 생각합니다.

교육과정 동안 프로젝트를 할 때마다 팀장을 맡아왔습니다.

할 때마다 느끼는 거지만, 인화(人和)라는 것은 참 어렵습니다.

나이 차이가 많이 나는, 그래서 가치관과 관점, 그리고 일에 임하는 자세도 많이 다른, 그런 사람들과 협력하는 것은 더욱 그렇습니다. 그럼에도 부족한 부분을 더 많은 시간을 들여, 더 많은 정성을 들여 채워 나가며 이렇게 소감이나마 남길 수 있어 다행입니다.

스스로를 게으른 편이라 생각했던 제가 성실히 일할 수 있는 역량을 기른 것 역시 이번 프로젝트의 성과로 거두어 갑니다.

프로젝트 완성을 위해 도움을 주신 팀원들에게도 감사를 표합니다.

감사합니다

GitHub 주소

https://github.com/jpminlak/Al_bracket2/tree/jpminlak

부록

(별첨 파일 참조)